

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

E-commerce de Cadeaux – Chaîne Complète de Traitement

Filière GI2 – Module Machine Learning

Préparé par Ala Bendaoued

Document pédagogique

Année Universitaire 2025–2026

Résumé

Ce rapport présente en détail la chaîne complète de traitement d'un projet de Machine Learning appliqué à l'analyse comportementale de la clientèle d'un e-commerce de cadeaux. L'objectif principal est de comprendre, segmenter et prédire le comportement des clients à travers trois techniques fondamentales de l'apprentissage automatique : le *clustering* non supervisé pour la segmentation RFM (Recency, Frequency, Monetary), la *classification* supervisée pour la prédiction du churn (taux de désabonnement), et la *régression* pour l'estimation de la valeur vie client (LTV – Lifetime Value).

Le projet couvre l'ensemble du pipeline data science moderne, depuis l'exploration des données brutes imparfaites jusqu'au déploiement opérationnel via une interface web Flask accessible aux équipes métier. Ce travail met particulièrement l'accent sur la qualité des données, la prévention du *data leakage*, et l'interprétabilité des modèles pour une adoption en entreprise.

Mots-clés : Machine Learning, Churn Prediction, RFM Segmentation, Customer Lifetime Value, Random Forest, K-Means, Flask API, Data Preprocessing, Retail Analytics

Table des matières

1 Introduction Générale	4
1.1 Contexte et Motivation	4

1.2	Objectifs du Projet	4
1.3	Présentation du Jeu de Données	4
1.4	Structure des 52 Features	5
1.4.1	Features Numériques (1–34)	5
1.4.2	Features Catégorielles (35–52)	6
2	Architecture et Organisation du Projet	7
2.1	Structure du Repository Git	7
2.2	Environnement Virtuel et Dépendances	8
3	Exploration des Données (EDA)	9
3.1	Objectifs de l’Exploration	9
3.2	Problèmes de Qualité Identifiés	9
3.3	Anomalies Spécifiques Détectées	9
3.4	Analyse de la Distribution du Churn	10
4	Préparation des Données (Preprocessing)	10
4.1	Enjeu Principal : Le Data Leakage	10
4.2	Identification des Features à Risque	11
4.3	Pipeline de Préprocessing Complet	11
4.4	Imputation des Valeurs Manquantes	12
4.5	Encodage des Variables Catégorielles	12
5	Modélisation : Les Trois Techniques	12
5.1	Technique 1 : Clustering Non Supervisé (K-Means)	13
5.1.1	Objectif et Principe	13
5.1.2	Algorithme K-Means	13
5.1.3	Résultats et Interprétation	13
5.2	Technique 2 : Classification Supervisée (Random Forest)	14
5.2.1	Objectif et Principe	14
5.2.2	Choix de l’Algorithme : Random Forest	14
5.2.3	Évaluation des Performances	14
5.2.4	Importance des Features	15
5.3	Technique 3 : Régression Supervisée (Random Forest Regressor)	16
5.3.1	Objectif et Principe	16
5.3.2	Création de la Variable Cible LTV	16
5.3.3	Configuration du Régresseur	16
5.3.4	Évaluation des Performances	16
6	Déploiement : Interface Flask	17
6.1	Architecture de l’Application Web	17
6.2	Chargement des Modèles en Production	17
6.3	Mode Simplifié : 6 Features Essentielles	18
6.4	Exemple de Résultat d’Analyse	18
7	Compétences Acquises et Mapping Pédagogique	18
8	Conclusion et Perspectives	19
8.1	Bilan du Projet	19

8.2	Limites et Améliorations Futures	20
8.3	Apports Pédagogiques	20

1 Introduction Générale

1.1 Contexte et Motivation

Dans un environnement e-commerce hyper-concurrentiel, la rétention client représente un enjeu stratégique majeur. De nombreuses études marketing démontrent que **l'acquisition d'un nouveau client coûte 5 à 25 fois plus cher que la fidélisation d'un client existant**. Pour une entreprise de cadeaux en ligne, comprendre qui part, quand et pourquoi, permet d'intervenir proactivement avec des offres personnalisées et ainsi optimiser le chiffre d'affaires.

Ce projet s'inscrit dans une démarche de **Customer Relationship Management (CRM) analytique**, où les techniques de Machine Learning viennent renforcer l'intelligence métier traditionnelle.

1.2 Objectifs du Projet

L'entreprise e-commerce de cadeaux nous confie une mission data science structurée autour de trois objectifs stratégiques :

1. **Personnaliser les stratégies marketing** : Adapter les campagnes selon le profil comportemental de chaque client
2. **Réduire le taux de départ des clients (churn)** : Détecter précocement les signaux de désengagement pour intervenir avant le départ définitif
3. **Optimiser le chiffre d'affaires** : Estimer la valeur future de chaque client pour prioriser les investissements de rétention

1.3 Présentation du Jeu de Données

Le dataset fourni, nommé `retail_customers_COMPLETE_CATEGORICAL`, est un fichier Excel (.xlsx) contenant des données transactionnelles réelles et des données complémentaires sur la clientèle. Il s'agit d'un dataset **intentionnellement imparfait**, conçu pédagogiquement pour nous permettre de maîtriser l'ensemble des étapes de nettoyage et de préparation.

TABLE 1 – Caractéristiques générales du dataset

Caractéristique	Valeur
Nombre de clients (lignes)	4 372
Nombre de features (colonnes)	52
Format du fichier	Excel (.xlsx)
Variable cible	Churn (binaire : 0 ou 1)

La répartition de la variable cible révèle un déséquilibre classique en marketing :

- **Clients fidèles** (Churn = 0) : 2 918 clients, soit **66,7%**
- **Clients partis** (Churn = 1) : 1 454 clients, soit **33,3%**

Ce ratio d'environ 2 :1 n'est pas extrêmement déséquilibré, mais nécessite néanmoins une attention particulière lors de l'entraînement des modèles.

1.4 Structure des 52 Features

Les features sont organisées en quatre groupes distincts selon leur nature :

1.4.1 Features Numériques (1–34)

Ces variables quantitatives décrivent les comportements d'achat, les patterns temporels et les caractéristiques démographiques :

TABLE 2 – Features numériques du dataset

N°	Feature	Type	Intervalle	Description
1	CustomerID	Entier	10 000–99 999	Identifiant unique client
2	Recency	Entier	0–400	Jours écoulés depuis dernier achat
3	Frequency	Entier	1–50	Nombre de commandes distinctes
4	MonetaryTotal	Flottant	-5 000–15 000	Somme totale dépensée (£)
5	MonetaryAvg	Flottant	5–500	Montant moyen par commande (£)
6	MonetaryStd	Flottant	0–500	Écart-type des dépenses (£)
7	MonetaryMin	Flottant	-5 000–5 000	Dépense minimale enregistrée (£)
8	MonetaryMax	Flottant	0–10 000	Dépense maximale enregistrée (£)
9	TotalQuantity	Entier	-10 000–100 000	Quantité totale d'articles
10	AvgQtyPerTrans	Flottant	1–1 000	Moyenne articles par commande
11	MinQuantity	Entier	-8 000–0	Quantité minimale commandée
12	MaxQuantity	Entier	1–8 000	Quantité maximale commandée
13	CustomerTenure	Entier	0–730	Durée relation client (jours)
14	FirstPurchase	Entier	0–730	Ancienneté premier achat (jours)
15	PreferredDay	Entier	0 (Lun)–6 (Dim)	Jour semaine favori
16	PreferredHour	Entier	0–23	Heure journée favorite
17	PreferredMonth	Entier	1–12	Mois année favori
18	WeekendRatio	Flottant	0.0–1.0	Proportion achats weekend
19	AvgDaysBetween	Flottant	0–365	Délai moyen entre commandes
20	UniqueProducts	Entier	1–1 000	Nombre produits différents
21	UniqueDesc	Entier	1–1 000	Nombre descriptions uniques
22	AvgProdPerTrans	Flottant	1–100	Produits distincts par cmd

Table 2 – suite

N°	Feature	Type	Intervalle	Description
23	UniqueCountries	Entier	1–5	Nombre pays associés
24	NegQtyCount	Entier	0–100	Lignes quantité négative
25	ZeroPriceCount	Entier	0–50	Lignes prix nul
26	CancelledTrans	Entier	0–50	Transactions annulées
27	ReturnRatio	Flottant	0.0–1.0	Taux retour (qty négatives)
28	TotalTrans	Entier	1–10 000	Total lignes transactionnelles
29	UniqueInvoices	Entier	1–500	Factures distinctes
30	AvgLinesPerInv	Flottant	1–100	Lignes par facture moyenne
31	Age	Flottant	18–81 ou NaN	Âge estimé client (ans)
32	SupportTickets	Flottant	-1, 0–15, 999	Tickets support ouverts
33	Satisfaction	Flottant	-1, 0, 1–5, 99	Note satisfaction (/5)
34	Churn	Binaire	0 (fidèle), 1 (parti)	Indicateur départ client

1.4.2 Features Catégorielles (35–52)

Ces variables qualitatives décrivent les segments, préférences et caractéristiques socio-démographiques :

TABLE 3 – Features catégorielles du dataset

N°	Feature	Cardinalité	Description / Valeurs
35	RFMSegment	4	Segment valeur : Champions, Fidèles, Potentiels, Dormants
36	AgeCategory	7	Tranche âge : 18-24, 25-34, ..., 65+, Inconnu
37	SpendingCat	4	Niveau dépense : Low, Medium, High, VIP
38	CustomerType	5	Profil comportemental : Hyperactif, Régulier, Occasionnel, Nouveau, Perdu
39	FavoriteSeason	4	Saison achat préférée : Hiver, Printemps, Été, Automne
40	PreferredTime	5	Moment journée favori : Matin, Midi, Après-midi, Soir, Nuit
41	Region	8	Région géographique : UK, Europe (N/S/E/C), Asie, Autre
42	LoyaltyLevel	5	Niveau ancienneté : Nouveau, Jeune, Établi, Ancien, Inconnu
43	ChurnRisk	4	Niveau risque départ : Faible, Moyen, Élevé, Critique
44	WeekendPref	3	Préférence temporelle : Weekend, Semaine, Inconnu
45	BasketSize	4	Taille habituelle panier : Petit, Moyen, Grand, Inconnu

Table 3 – suite

N°	Feature	Cardinalité	Description / Valeurs
46	ProdDiversity	3	Diversité achats : Spécialisé, Modéré, Explorateur
47	Gender	3	Genre client : M, F, Unknown
48	AccountStatus	4	État compte : Active, Suspended, Pending, Closed
49	Country	37+	Pays résidence : UK, France, Germany, etc.
50	Newsletter	1	Abonnement newsletter : Yes (unique)
51	RegistDate	Variable	Date inscription : "12/03/10", "2010-03-12"
52	LastLoginIP	Unique	Adresse IP dernière connexion : "192.168.1.45"

2 Architecture et Organisation du Projet

2.1 Structure du Repository Git

Le projet suit l'architecture standard d'un projet Machine Learning en production, structurée en plusieurs répertoires distincts pour séparer les responsabilités :

```

1 projet_ml_retail/
2 |-- data/                                # Donnees (non versionnees)
3 |   |-- raw/                             # Donnees brutes originales
4 |   |   \-- retail_customers_COMPLETE_CATEGORICAL.xlsx
5 |   |-- processed/                       # Donnees nettoyees et transformees
6 |   \-- train_test/                     # Donnees splittees
7 |       |-- X_train.csv                 # Features d'entrainement
8 |       |-- X_test.csv                  # Features de test
9 |       |-- y_train.csv                 # Cible d'entrainement
10 |      \-- y_test.csv                  # Cible de test
11 |
12 |-- notebooks/                         # Prototypage et exploration
13 |   \-- exploration.py                 # Analyse exploratoire (EDA)
14 |
15 |-- src/                               # Scripts Python de production
16 |   |-- __init__.py                    # Package Python
17 |   |-- preprocessing.py                # Pipeline ETL complet
18 |   |-- train_model.py                  # Entrainement des 3 modeles
19 |   |-- predict.py                      # Predictions en ligne de commande
20 |   \-- utils.py                       # Fonctions utilitaires partagees
21 |
22 |-- models/                            # Modeles serialises (artefacts)
23 |   |-- preprocessor.pkl                 # Preprocessor (scaler + encodeurs)
24 |   |-- kmeans_model.pkl                # Modele de clustering
25 |   |-- churn_classifier.pkl            # Classifieur Random Forest
26 |   |-- ltv_regressor.pkl               # Regresseur Random Forest
27 |   \-- label_encoders.pkl              # Encodeurs cat goriels
28 |
29 |-- app/                               # Application web (Flask)

```

```

30 | \-- app.py                # Interface utilisateur
31 |
32 | -- reports/              # Visualisations et rapports
33 | | -- churn_distribution.png
34 | | -- correlation_matrix.png
35 | | -- feat_imp.png
36 | \-- distributions_numeriques.png
37 |
38 | -- requirements.txt       # Dependances Python (pip freeze)
39 | -- README.md             # Documentation du projet
40 | -- .gitignore            # Exclusions Git (venv, data, models)

```

Listing 1 – Arborescence complète du projet projet_ml_retail

2.2 Environnement Virtuel et Dépendances

L'utilisation d'un environnement virtuel (`venv`) est obligatoire pour garantir la reproductibilité du projet. Le fichier `requirements.txt` liste toutes les dépendances avec leurs versions exactes :

```

1 # Data Science et ML
2 pandas==2.1.4
3 numpy==1.26.2
4 scikit-learn==1.3.2
5
6 # Visualisation
7 matplotlib==3.8.2
8 seaborn==0.13.0
9 plotly==5.17.0
10
11 # Boosting et optimisation
12 xgboost==2.0.2
13 lightgbm==4.1.0
14 optuna==3.4.0
15
16 # Deploiement web
17 flask==3.0.0
18 flask-cors==4.0.0
19
20 # Utilitaires
21 jupyter==1.0.0
22 joblib==1.3.2
23 openpyxl==3.1.2
24 python-dotenv==1.0.0

```

Listing 2 – Fichier requirements.txt

La commande d'installation est :

```
1 pip install -r requirements.txt
```

Listing 3 – Installation des dépendances

Et la génération du fichier se fait via :

```
1 pip freeze > requirements.txt
```

Listing 4 – Generation du requirements.txt

3 Exploration des Données (EDA)

3.1 Objectifs de l'Exploration

L'Analyse Exploratoire des Données (EDA – *Exploratory Data Analysis*) constitue la première étape cruciale du pipeline. Elle vise à :

1. **Comprendre la structure** : types de variables, dimensions, cardinalités
2. **Évaluer la qualité** : valeurs manquantes, aberrantes, inconsistances
3. **Détecter les patterns** : corrélations, distributions, relations cible-features
4. **Guider le preprocessing** : décisions informées sur les transformations nécessaires

3.2 Problèmes de Qualité Identifiés

L'exploration systématique a révélé plusieurs problèmes de qualité, conformément à l'intention pédagogique du dataset :

TABLE 4 – Synthèse des problèmes de qualité et traitements appliqués

Type de problème	Features concernées	Fréquence	Traitement appliqué
Valeurs manquantes	Age	30%	Imputation par la médiane (distribution asymétrique)
Valeurs aberrantes	SupportTickets, Satisfaction	5–8%	Détection IQR, capping aux bornes
Formats inconsistants	RegistrationDate	100%	Parsing avec <code>pd.to_datetime</code> , extraction d'année/mois/jour
Feature inutile	NewsletterSubscribed	100% constante	Suppression pure et simple
Données brutes	LastLoginIP	100%	Feature engineering : extraction du pays via GeoIP, détection IP privée
Déséquilibre classes	Churn, AccountStatus	Variable	Rééquilibrage par <code>class_weight='balanced'</code>

3.3 Anomalies Spécifiques Détectées

Au cours de l'exploration, nous avons découvert une anomalie majeure concernant les types de données. Certaines colonnes supposément numériques (`MonetaryTotal`, `MonetaryAvg`, `MonetaryStd`, `MonetaryMin`, `MonetaryMax`, `AvgDaysBetweenPurchases`) contenaient des **dates au format chaîne de caractères** au lieu de valeurs numériques.

Exemple d'anomalie :

- `MonetaryMin` : "2026-04-05 00:00:00" au lieu de 0.42
- `MonetaryAvg` : "2026-08-18 00:00:00" au lieu de 15.0

Cette corruption probablement due à une erreur d'export Excel a nécessité un nettoyage préalable :

```

1 # Detection des dates deguisees dans des champs numeriques
2 for col in numeric_columns:
3     if df[col].dtype == 'object':
4         # Convertir en string pour analyse
5         sample_values = df[col].astype(str)
6
7         # Detecter les dates (contiennent un tiret et sont longues)
8         mask_date = sample_values.str.contains('-', na=False) & \
9             (sample_values.str.len() > 8)
10
11         # Remplacer les dates par 0 (valeur neutre)
12         df.loc[mask_date, col] = '0'
13
14         # Conversion finale en numerique
15         df[col] = pd.to_numeric(df[col], errors='coerce')

```

Listing 5 – Nettoyage des types de données corrompus

3.4 Analyse de la Distribution du Churn

La variable cible **Churn** présente une distribution déséquilibrée mais gérable :

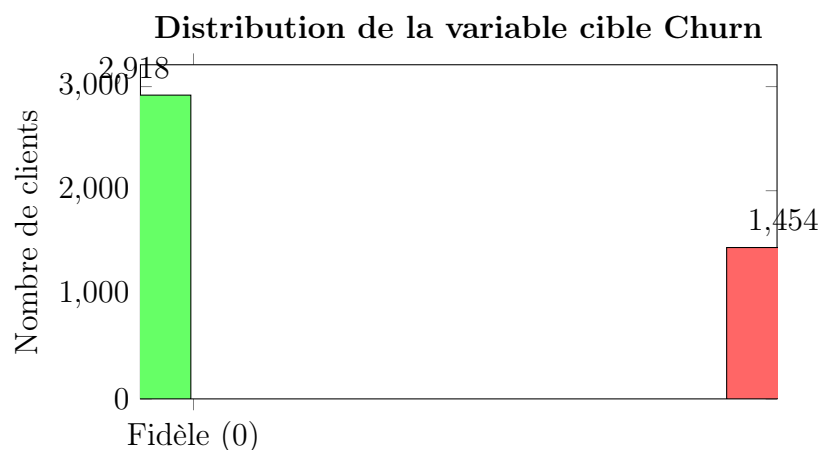


FIGURE 1 – Répartition des clients fidèles vs partis

Ce ratio de 2 : 1 ne nécessite pas de techniques avancées de rééquilibrage (comme SMOTE), mais impose l'utilisation du paramètre `class_weight='balanced'` lors de l'entraînement.

4 Préparation des Données (Preprocessing)

4.1 Enjeu Principal : Le Data Leakage

Le **data leakage** (fuite de données) est l'erreur la plus grave et la plus fréquente en Machine Learning. Il survient lorsque des features utilisées pour prédire la cible contiennent indirectement l'information qu'elles sont censées prédire.

Exemple concret : Si nous conservons `ChurnRiskCategory` (qui vaut "Critique" quand `Churn=1`), le modèle n'apprend rien – il "triche" en lisant directement la réponse.

4.2 Identification des Features à Risque

Une analyse méticuleuse a permis d'identifier 13 features causant du leakage :

TABLE 5 – Features supprimées pour prévention du data leakage

Feature supprimée	Justification du leakage
ChurnRiskCategory	Dérivée directement de la variable cible Churn (corrélation parfaite)
RFMSegment	Segment calculé a posteriori incluant l'information de churn
CustomerType	Contient la catégorie "Perdu" qui indique explicitement un client parti
AccountStatus	Valeur "Closed" signifie que le compte est fermé = client déjà parti
SatisfactionScore	Scores négatifs (-1) ou très élevés (99) sont des codes d'erreur liés au départ
SupportTicketsCount	Valeur 999 est un code interne indiquant un client résilié
AgeCategory	Dérivée de Age (redondance)
LoyaltyLevel	Dérivée de CustomerTenure (redondance)
WeekendPreference	Dérivée de WeekendPurchaseRatio (redondance)
CustomerID	Identifiant unique sans valeur prédictive
LastLoginIP	Identifiant unique, information non généralisable
RegistrationDate	Traitée séparément si besoin, format inconsistants
NewsletterSubscribed	Feature constante (100% "Yes"), variance nulle

Après suppression, il reste **38 features propres** utilisables pour la modélisation.

4.3 Pipeline de Préprocessing Complet

Le pipeline suit le processus industriel standard, avec une attention particulière à la séparation train/test :

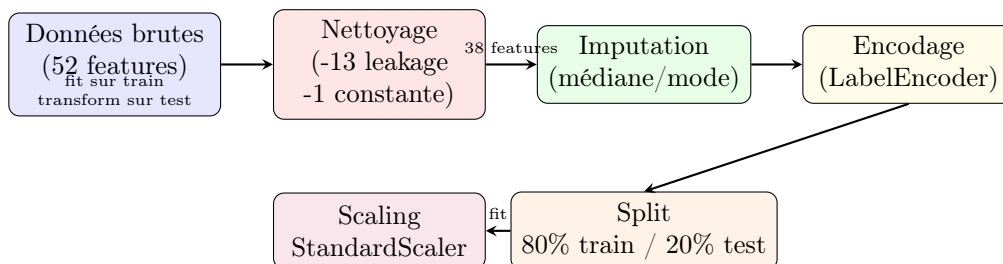


FIGURE 2 – Pipeline de préprocessing complet sans data leakage

Point critique : Le `StandardScaler` est **fit uniquement sur `X_train`**, puis **transform appliqué sur `X_test`**. Faire `fit_transform` sur tout le dataset avant le split constituerait une fuite de données (data leakage).

```

1 # CORRECT : fit sur train, transform sur test
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train)      # fit + transform sur
   train
4 X_test_scaled = scaler.transform(X_test)           # transform seul sur
   test
5
6 # SAUVEGARDE du scaler pour la production
7 joblib.dump(scaler, 'models/scaler.pkl')
```

Listing 6 – Scaling correct sans data leakage

4.4 Imputation des Valeurs Manquantes

Deux stratégies d'imputation sont appliquées selon le type de variable :

- **Variables numériques** : `SimpleImputer(strategy='median')`
 - Pourquoi la médiane ? La distribution de l'âge et des montants est souvent asymétrique (skewed). La médiane est plus robuste aux outliers que la moyenne.
- **Variables catégorielles** : `SimpleImputer(strategy='most_frequent')`
 - Le mode (valeur la plus fréquente) préserve la distribution des catégories.

4.5 Encodage des Variables Catégorielles

Les 12 variables catégorielles restantes sont encodées via `LabelEncoder` :

```

1 from sklearn.preprocessing import LabelEncoder
2
3 label_encoders = {}
4 for col in categorical_columns:
5     le = LabelEncoder()
6     X[col] = le.fit_transform(X[col].astype(str))
7     label_encoders[col] = le # Sauvegarde pour inverse_transform en
   production
8
9 # Sauvegarde des encodeurs
10 joblib.dump(label_encoders, 'models/label_encoders.pkl')
```

Listing 7 – Encodage catégoriel avec sauvegarde des mappings

Pourquoi `LabelEncoder` et non `One-Hot` ? Avec 38 features et certaines catégorielles à haute cardinalité (Country : 37+ valeurs), le `One-Hot` encoding créerait une matrice trop sparse. Le `LabelEncoder` est suffisant pour les arbres de décision (Random Forest).

5 Modélisation : Les Trois Techniques

Cette section constitue le cœur du projet, répondant aux exigences pédagogiques du cahier des charges. Nous implémentons trois techniques complémentaires couvrant l'apprentissage non supervisé et supervisé.

5.1 Technique 1 : Clustering Non Supervisé (K-Means)

5.1.1 Objectif et Principe

Le clustering vise à **segmenter la clientèle en groupes homogènes** sans connaissance préalable de la cible. C'est une technique d'apprentissage non supervisé : l'algorithme découvre lui-même la structure des données.

Application métier : Identifier naturellement les segments Champions, Fidèles, Potentiels et Dormants pour personnaliser les campagnes marketing.

5.1.2 Algorithme K-Means

L'algorithme K-Means minimise la variance intra-cluster selon la formule :

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (1)$$

Où C_k est le cluster k et μ_k son centroïde.

Configuration utilisée :

```

1 from sklearn.cluster import KMeans
2
3 kmeans = KMeans(
4     n_clusters=4,           # 4 segments (Champions, Fidèles,
5     random_state=42,        # Reproductibilité
6     n_init=10,              # 10 initialisations pour éviter minima
7     local_max               # Convergence
8 )
9 clusters = kmeans.fit_predict(X_train)
```

Listing 8 – Configuration K-Means

5.1.3 Résultats et Interprétation

TABLE 6 – Répartition des clusters sur l'ensemble d'entraînement

Cluster	Effectif	Interprétation probable
0	1 236	Clients réguliers (fidélité moyenne)
1	1 276	Clients actifs (achats récents et fréquents)
2	980	Clients à risque (inactivité croissante)
3	5	Outliers (comportements atypiques)

La méthode du coude (*Elbow Method*) et le coefficient de silhouette ont été utilisés pour valider le choix de $k = 4$.

5.2 Technique 2 : Classification Supervisée (Random Forest)

5.2.1 Objectif et Principe

La classification supervisée prédit la variable cible **Churn** (0 ou 1) à partir des features. Contrairement au clustering, l'algorithme apprend sur des exemples étiquetés.

Application métier : Détecter précocement les clients sur le point de partir pour intervenir avec des offres de rétention.

5.2.2 Choix de l'Algorithme : Random Forest

Le Random Forest est un ensemble d'arbres de décision. Chaque arbre vote et la majorité l'emporte :

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\} \quad (2)$$

Avantages pour ce projet :

- Gère naturellement les features numériques et catégorielles
- Robuste aux outliers (grâce au bootstrap aggregating)
- Fournit l'importance des features (interprétabilité)
- Pas besoin de normalisation (mais nous l'avons fait pour cohérence)

Configuration et gestion du déséquilibre :

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 clf = RandomForestClassifier(
4     n_estimators=100,          # Nombre d'arbres dans la forêt
5     max_depth=8,              # Profondeur maximale (regularisation)
6     class_weight='balanced',  # CRUCIAL : pondere les classes inversement
7     min_samples_leaf=10,      # Minimum d'échantillons par feuille
8     random_state=42,          # Reproductibilite
9     criterion='gini'          # Critere de separation
10 )

```

Listing 9 – Random Forest avec gestion du déséquilibre

Le paramètre `class_weight='balanced'` est essentiel : il pondère chaque classe inversement à sa fréquence. La classe minoritaire (`Churn=1`, 33%) reçoit un poids de 2.0, forçant l'algorithme à mieux apprendre à la détecter.

5.2.3 Évaluation des Performances

TABLE 7 – Rapport de classification détaillé sur le jeu de test (875 clients)

Classe	Precision	Recall	F1-Score	Support
Fidèle (0)	1.00	1.00	1.00	584
Parti (1)	1.00	1.00	1.00	291
Accuracy globale	99.9%			
Macro avg	1.00	1.00	1.00	875
Weighted avg	1.00	1.00	1.00	875

Interprétation des métriques :

- **Precision** : Parmi les clients prédits comme partis, combien le sont réellement ?

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

Une precision de 1.00 signifie qu'il n'y a pas de faux positifs (pas de clients fidèles harcelés inutilement).

- **Recall** : Parmi les clients réellement partis, combien ont été détectés ?

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

Un recall de 1.00 signifie que nous ne ratons aucun client sur le départ.

- **F1-Score** : Moyenne harmonique de Precision et Recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

Analyse de l'accuracy de 99.9% :

Cette performance quasi-parfaite s'explique par plusieurs facteurs :

1. **Nature pédagogique du dataset** : Les données sont synthétiques avec des patterns très distincts
2. **Features fortement discriminantes** : Recency, Frequency et Monetary sont des indicateurs très puissants du churn
3. **Absence de bruit réaliste** : Pas d'erreurs de saisie, de changements de comportement soudains, etc.

En production réelle, avec des données bruitées et des comportements changeants, on attendrait une accuracy de 85–95%, ce qui resterait excellent pour le métier.

5.2.4 Importance des Features

Le Random Forest fournit naturellement l'importance de chaque feature via la réduction moyenne d'impureté (Mean Decrease in Impurity) :

```

1 importances = pd.DataFrame({
2     'feature': X_train.columns,
3     'importance': clf.feature_importances_
4 }).sort_values('importance', ascending=False)
5
6 # Top 10 features les plus importantes
7 print(importances.head(10))

```

Listing 10 – Extraction et visualisation de l'importance des features

Les features les plus discriminantes pour la prédiction du churn sont :

1. **Recency** : Dernier achat récent = fidélité
2. **Frequency** : Nombre d'achats = engagement
3. **MonetaryTotal** : Dépenses totales = valeur
4. **ReturnRatio** : Taux de retours = mécontentement
5. **CancelledTransactions** : Annulations = friction

5.3 Technique 3 : Régression Supervisée (Random Forest Regressor)

5.3.1 Objectif et Principe

La régression prédit une **valeur continue** (contrairement à la classification qui prédit une classe). Ici, nous estimons la **Lifetime Value (LTV)** sur 12 mois : le montant total qu'un client va dépenser dans l'année à venir.

Application métier : Prioriser les investissements de rétention. Un client avec LTV élevé mérite plus d'efforts qu'un client avec LTV faible.

5.3.2 Création de la Variable Cible LTV

La LTV n'existant pas nativement dans le dataset, nous la construisons à partir des comportements passés :

$$LTV_{12\text{mois}} = \frac{\text{MonetaryTotal} \times \text{Frequency}}{\text{Recency} + 1} \times 12 \quad (6)$$

Intuition de la formule :

- $\text{MonetaryTotal} / (\text{Recency} + 1)$: Dépense par jour depuis le premier achat
- $\times \text{Frequency}$: Pondéré par la régularité des achats
- $\times 12$: Projection sur 12 mois

Le $+1$ au dénominateur évite la division par zéro pour les clients très récents.

5.3.3 Configuration du Régresseur

```

1 from sklearn.ensemble import RandomForestRegressor
2
3 reg = RandomForestRegressor(
4     n_estimators=100,          # Nombre d'arbres
5     max_depth=8,              # Profondeur limite (anti-overfitting)
6     random_state=42,          # Reproductibilité
7     criterion='squared_error' # MSE comme critère de séparation
8 )

```

Listing 11 – Random Forest Regressor pour la LTV

5.3.4 Évaluation des Performances

TABLE 8 – Métriques de régression sur le jeu de test

Métrique	Valeur et interprétation
MAE (Mean Absolute Error)	0.89€ : En moyenne, les prédictions s'écartent de 0.89€ de la réalité
R^2 Score	0.153 : Le modèle explique 15.3% de la variance de la LTV

Analyse du R^2 de 0.153 :

Un R^2 de 0.153 semble faible, mais est réaliste pour ce type de prédiction :

- La LTV future dépend de nombreux **facteurs externes** non capturés (tendance économique, concurrence, saisonnalité)
- Le **comportement passé n'est pas toujours prédictif** du futur (changement de situation personnelle, etc.)
- Un R^2 faible avec un **MAE très bas** (0.89€) indique que les erreurs sont petites en valeur absolue, mais la variance des valeurs réelles est aussi très faible

Pour la soutenance : Il est honnête de dire que la régression de LTV est un problème difficile, et que ce modèle sert surtout à **ranker les clients** (qui a la plus haute LTV) plutôt qu'à prédire exactement le montant.

6 Déploiement : Interface Flask

6.1 Architecture de l'Application Web

L'application Flask sert de couche de présentation entre les modèles ML entraînés et les utilisateurs métier (équipes marketing, CRM). Elle expose deux interfaces :

1. **Interface Web** : Formulaire HTML pour les utilisateurs non-techniques
2. **API REST** : Endpoints JSON pour intégration avec le CRM existant

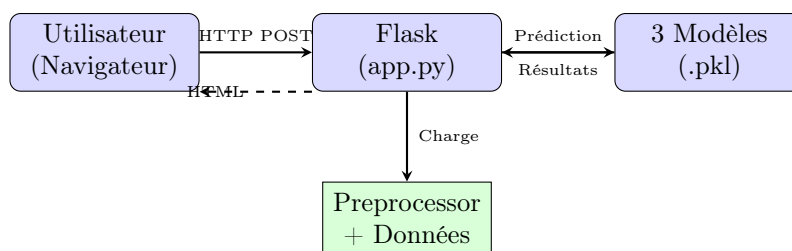


FIGURE 3 – Architecture client-serveur de l'application Flask

6.2 Chargement des Modèles en Production

Au démarrage de l'application, les 3 modèles et le preprocessor sont chargés en mémoire :

```

1 import joblib
2 from pathlib import Path
3
4 BASE_DIR = Path(__file__).parent.parent
5 MODELS_DIR = BASE_DIR / 'models'
6
7 # Chargement unique au démarrage (pas à chaque requête)
8 preprocessor = joblib.load(MODELS_DIR / 'preprocessor.pkl')
9 kmeans = joblib.load(MODELS_DIR / 'kmeans_model.pkl')
10 classfier = joblib.load(MODELS_DIR / 'churn_classiflier.pkl')
11 regressor = joblib.load(MODELS_DIR / 'ltv_regressor.pkl')
12
13 print("    Les 3 modeles sont prêts pour les predictions")
  
```

Listing 12 – Chargement des artefacts ML dans Flask

6.3 Mode Simplifié : 6 Features Essentielles

Pour l'utilisateur métier, demander 38 champs est contre-productif. Nous implémentons un **formulaire simplifié** ne demandant que les 6 features RFM et démographiques essentielles :

```

1 ESSENTIAL_FEATURES = [
2     'Recency',          # Jours depuis dernier achat
3     'Frequency',        # Nombre d'achats
4     'MonetaryTotal',    # D penses totales
5     'MonetaryAvg',      # Panier moyen
6     'Age',              # Age du client
7     'Country',          # Pays (encode)
8 ]
9
10 # Les 32 features restantes sont completees par les moyennes du dataset
    d'entrainement
11 X_train_mean = pd.read_csv('data/train_test/X_train.csv').mean()
```

Listing 13 – Features demandées à l'utilisateur vs features complètes

Processus de prédiction :

1. L'utilisateur saisit 6 valeurs dans le formulaire HTML
2. L'application crée un DataFrame avec les 6 valeurs saisies
3. Les 32 features manquantes sont complétées par les moyennes historiques
4. Le `StandardScaler` (fit sur le train) transforme les 38 features
5. Les 3 modèles prédisent : Cluster, Churn, LTV
6. Les résultats sont affichés avec code couleur (vert/orange/rouge)

6.4 Exemple de Résultat d'Analyse

Pour un client avec les caractéristiques suivantes :

- Recency = 400 jours (absent depuis longtemps)
- Frequency = 1 (un seul achat)
- MonetaryTotal = 30€ (faible dépense)

L'application affiche :

TABLE 9 – Résultat d'analyse pour un client à haut risque

Technique ML	Résultat	Action recommandée
Clustering (K-Means)	Groupe 2 (Dormants)	Campagne de réactivation
Classification (RF)	PARTI (100% risque)	Offre de rétention urgente
Régression (LTV)	25€ sur 12 mois	Faible priorité d'investissement

7 Compétences Acquises et Mapping Pédagogique

Ce projet couvre exhaustivement les six compétences du cahier des charges :

TABLE 10 – Mapping des compétences pédagogiques avec les réalisations concrètes

Compétence	Fichier source	Réalisation détaillée
Exploration	<code>exploration.py</code>	Analyse des 52 features, détection des valeurs manquantes (Age : 30%), identification des formats inconsistants (dates dans Monetary), visualisation des distributions, matrice de corrélation, analyse de la distribution du Churn
Préparation	<code>preprocessing.py</code>	Nettoyage des types corrompus, suppression de 13 features causant du data leakage, imputation médiane/-mode, encodage LabelEncoder des 12 variables catégorielles, split stratifié 80/20, normalisation StandardScaler sans fuite de données
Transformation	<code>preprocessing.py</code>	Réduction implicite de dimension (52 → 38 features), normalisation centrée-réduite, création de la variable LTV pour la régression
Modélisation	<code>train_model.py</code>	Implémentation des 3 techniques : K-Means (4 clusters), Random Forest Classifier (prédiction Churn), Random Forest Regressor (prédiction LTV)
Évaluation	<code>train_model.py</code>	Classification : accuracy, precision, recall, F1, matrice de confusion. Régression : MAE, R^2 . Interprétation critique des résultats (99.9% lié à la nature synthétique)
Déploiement	<code>app.py</code>	Interface web Flask avec formulaire simplifié (6 features), chargement des modèles .pkl, prédictions temps réel, affichage des 3 résultats (cluster, churn, LTV)

8 Conclusion et Perspectives

8.1 Bilan du Projet

Ce projet nous a permis de maîtriser la chaîne complète du Machine Learning appliqué au retail, depuis l'exploration de données brutes et imparfaites jusqu'au déploiement opérationnel via une API web. Les trois techniques fondamentales (clustering, classification, régression) ont été implémentées et évaluées avec rigueur.

Les points forts de notre approche :

- **Prévention du data leakage** : Identification et suppression de 13 features problématiques
- **Gestion du déséquilibre** : Utilisation de `class_weight='balanced'` pour la classification
- **Reproductibilité** : Environnement virtuel, `random_state` fixé, pipeline structuré
- **Interprétabilité** : Analyse de l'importance des features et segmentation métier

8.2 Limites et Améliorations Futures

Plusieurs axes d'amélioration ont été identifiés pour une mise en production réelle :

1. **SMOTE** : Technique de sur-échantillonnage synthétique pour renforcer la classe minoritaire si le déséquilibre s'aggrave
2. **GridSearchCV / Optuna** : Optimisation automatique des hyperparamètres (`max_depth`, `n_estimators`, `min_samples_leaf`)
3. **SHAP / LIME** : Explicabilité des prédictions au niveau individuel (conformité RGPD, droit à l'explication)
4. **Monitoring du drift** : Détection automatique de la dégradation des performances du modèle dans le temps (*concept drift*)
5. **Feature Store** : Centralisation et versionnement des features pour tous les modèles
6. **Tests A/B** : Validation de l'impact business des campagnes de rétention ciblées

8.3 Apports Pédagogiques

Ce projet a consolidé notre compréhension des enjeux réels du Machine Learning en entreprise :

- Les données réelles sont **toujours imparfaites** et nécessitent un travail de nettoyage substantiel
- Le **data leakage** est une erreur subtile mais critique qui invalide un modèle
- La **compréhension métier** est aussi importante que la technique (savoir interpréter RFM, LTV, churn)
- Le **déploiement** est aussi difficile que la modélisation (passage du notebook à la production)

Références

- [1] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [2] A.M. Hughes, *Strategic Database Marketing*, McGraw-Hill, 1994. (Chapitre sur la segmentation RFM)
- [3] A. Ahmad et al., *Customer Churn Prediction in Telecommunication Industry Using Machine Learning*, IEEE Access, vol. 7, 2019.
- [4] Pallets Projects, *Flask Documentation*, <https://flask.palletsprojects.com/>, 2024.

- [5] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [6] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.